WEEK 7 PRACTICAL QUESTION 1

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Aim

To write a C program that displays the memory addresses of variables belonging to the code, global, static, heap, and stack segments and understand the Linux process address-space layout.

Program

```
 GNU nano 7.2                          memory_layout.c *
#include <stdio.h>
#include <stdlib.h>

// Global variable
int global_var = 10;

// Static variable
static int static_var = 20;

// Code segment
void code_function()
{
    printf("Code address            : %p\n", (void *)code_function);
}

int main()
{
    // Stack variable
    int stack_var = 30;

    // Heap variable
    int *heap_var = (int *)malloc(sizeof(int));
    *heap_var = 40;

    printf("Linux Process Address Space\n");
    printf("---------------------------\n");

    code_function();

    printf("Global variable address : %p\n", (void *)&global_var);
    printf("Static variable address : %p\n", (void *)&static_var);
    printf("Heap variable address   : %p\n", (void *)heap_var);
    printf("Stack variable address  : %p\n", (void *)&stack_var);

    free(heap_var);

    return 0;
}
```

Output

The exact addresses will be different on every execution because of Linux memory-layout randomization.

Memory Addresses:

```
suraj@DESKTOP-BLT5L3R:~$ cd ~/OSSP
suraj@DESKTOP-BLT5L3R:~/OSSP$ mkdir -p Week7
suraj@DESKTOP-BLT5L3R:~/OSSP$ cd Week7
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ pwd
/home/suraj/OSSP/Week7
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ nano memory_layout.c
vuraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ gcc memory_layout.c -o memory_layout
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ ./memory_layout
Linux Process Address Space
---------------------------
Code address            : 0x654170d891c9
Global variable address : 0x654170d8c010
Static variable address : 0x654170d8c014
Heap variable address   : 0x654195a942a0
Stack variable address  : 0x7ffe55a559ac
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$
```

Observation

The program displays the addresses of variables located in different parts of the process address space.

Memory Segment	Example	Purpose
Code / Text	sample_function	Contains executable program instructions
Global / Data	global_var	Stores initialized global variables
Static	static_global_var, static_local_var	Static variables have program-long lifetime and are typically stored in the data or BSS segment depending on initialization.
Heap	heap_var	Used for dynamically allocated memory using malloc()
Stack	stack_var	Stores local variables and function-call information

The addresses demonstrate that different types of data are maintained in different regions of the Linux process address space.

Result

The program was successfully implemented to print the addresses of **code, global, static, heap,** and **stack variables**. The Linux process memory layout was analyzed based on the addresses obtained.

QUESTION 2

Make use of the /proc/maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Aim

To study the memory organization of a running Linux process using the /proc//maps file and memory-analysis commands, and understand virtual memory mappings.

Program

```
  GNU nano 7.2                    memory_process.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;

int main()
{
    int stack_var = 20;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 30;

    printf("Process ID: %d\n", getpid());
    printf("Global variable address : %p\n", (void *)&global_var);
    printf("Heap variable address   : %p\n", (void *)heap_var);
    printf("Stack variable address  : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");
    printf("Press Enter to exit.\n");

    getchar();

    free(heap_var);

    return 0;
}
```

Output

```
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ nano memory_process.c
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ gcc memory_process.c -o memory_process
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ gcc memory_process.c -o memory_process
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ ./memory_process
Process ID: 834
Global variable address : 0x64d45fe75010
Heap variable address   : 0x64d4855d82a0
Stack variable address  : 0x7ffe8f41be9c

Process is running...
Press Enter to exit.
```

Useful Memory Analysis Commands

To examine the process mappings:

cat /proc/1722/maps

Sample /proc/1722/maps Output

```
araj@DESKTOP-BLT5L3R:~/OSSP/Week7$ cat /proc/1722/map
5e50db78000-55e50db79000 r--p 00000000 08:30 80251        /home/suraj/OSSP/Week7/memory_maps
5e50db79000-55e50db7a000 r-xp 00001000 08:30 80251        /home/suraj/OSSP/Week7/memory_maps
5e50db7a000-55e50db7b000 r--p 00002000 08:30 80251        /home/suraj/OSSP/Week7/memory_maps
5e50db7b000-55e50db7c000 rw-p 00003000 08:30 80251        /home/suraj/OSSP/Week7/memory_maps
5e533342000-55e533363000 rw-p 00000000 00:00 0            [heap]
f4b7ba60000-7f4b7ba628000 r--p 00000000 08:30 13647       /usr/lib/x86_64-linux-gnu/libc.so.6
f4b7ba628000-7f4b7ba7c0000 r-xp 00028000 08:30 13647      /usr/lib/x86_64-linux-gnu/libc.so.6
f4b7ba7c0000-7f4b7ba8e0000 r--p 001c0000 08:30 13647      /usr/lib/x86_64-linux-gnu/libc.so.6
f4b7ba8e0000-7f4b7ba812000 rw-p 002d0d000 08:30 13647     /usr/lib/x86_64-linux-gnu/libc.so.6
f4b7ba812000-7f4b7ba814000 rw-p 00211000 08:30 13647      /usr/lib/x86_64-linux-gnu/libc.so.6
f4b7ba814000-7f4b7ba821000 rw-p 00000000 00:00 0
f4b7ba8df000-7f4b7ba8e2000 rw-p 00000000 00:00 0
f4b7ba8e2000-7f4b7ba8ee000 rw-p 00000000 00:00 0
f4b7ba8ee000-7f4b7ba8f0000 rw-p 00000000 00:00 0
f4b7ba8f0000-7f4b7ba8f2000 rw-p 00000000 00:00 0          /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
f4b7ba8f3000-7f4b7ba920000 r-xp 00001000 08:30 13644      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
f4b7ba920000-7f4b7ba92d000 r--p 00030000 08:30 13644      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
f4b7ba92d000-7f4b7ba93d000 rw-p 0003d000 08:30 13644      /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
f4b7ba93d000-7f4b7ba931000 rw-p 00000000 00:00 0
ffd2f6560000-7fd2f65530000 rw-p 00000000 00:00 0
uraj@DESKTOP-BLT5L3R:~/OSSP/Week7$
```

To view memory statistics:

cat /proc/1722/status

```
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ cat /proc/1722/status
Name:        memory_maps
Umask:       0022
State:       S (sleeping)
Tgid:        1722
Ngid:        0
Pid:         1722
PPid:        793
TracerPid:   0
Uid:         1000    1000    1000    1000
Gid:         1000    1000    1000    1000
FDSize:      256
Groups:      4 24 27 30 46 100 1000
NStgid:      1722
NSpid:       1722
NSpgid:      1722
NSsid:       793
Kthread:     0
VmPeak:        2768 kB
VmSize:        2768 kB
VmLck:            0 kB
```

To view detailed memory information:

cat /proc/1722/smaps

```
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ pmap 1722
1722:    ./memory_maps
000055e50db78000      4K r----   memory_maps
000055e50db79000      4K r-x--   memory_maps
000055e50db7a000      4K r----   memory_maps
000055e50db7b000      4K rw---   memory_maps
000055e535342000    132K rw---   [ anon ]
00007f4a7ba60c000   160K r----   libc.so.6
00007f4a7ba628000 1632K r-x--   libc.so.6
00007f4a7ba7c0000   312K r----   libc.so.6
00007f4a7ba8e0000    16K r----   libc.so.6
00007f4a7ba812000     8K rw---   libc.so.6
00007f4a7ba814000    52K rw---   [ anon ]
00007f4a7ba8df000    12K rw---   [ anon ]
00007f4a7ba8e8000     8K rw---   [ anon ]
00007f4a7ba8ea000    16K r----   [ anon ]
00007f4a7ba8ee000     8K r----   [ anon ]
00007f4a7ba8f0000     8K r-x--   [ anon ]
00007f4a7ba8f2000     4K r----   ld-linux-x86-64.so.2
00007f4a7ba8f3000   188K r-x--   ld-linux-x86-64.so.2
00007f4a7ba922000    44K r----   ld-linux-x86-64.so.2
00007f4a7ba92d000     8K r----   ld-linux-x86-64.so.2
00007f4a7ba92f000     4K rw---   ld-linux-x86-64.so.2
00007ffd2f630000    132K rw---   [ stack ]
total             2768K
```

You can also use:

pmap 1722

```
suraj@DESKTOP-BLT5L3R:~/OSSP/Week7$ pmap 1722
1722:    ./memory_process
000055e50db78000      4K  r----   memory_process
000055e50db79000      4K  r-x--   memory_process
000055e50db7a000      4K  r----   memory_process
000055e50db7b000      4K  rw---   memory_process
00007f4a7ba60000    132K  rw---   [ anon ]
00007f4a7ba62000    160K  r----   libc.so.6
00007f4a7ba7c000   1632K  r-x--   libc.so.6
00007f4a7ba8e000    312K  r----   libc.so.6
00007f4a7ba8f000     16K  r----   libc.so.6
00007f4a7ba92000      8K  rw---   libc.so.6
00007f4a7ba94000     52K  rw---   [ anon ]
00007f4a7ba8df000    12K  rw---   [ anon ]
00007f4a7ba8e8000     8K  rw---   [ anon ]
00007f4a7ba8ea000    16K  r----   [ anon ]
00007f4a7ba8ee000     8K  r----   [ anon ]
00007f4a7ba8f0000     8K  r-x--   [ anon ]
00007f4a7ba8f2000     4K  r----   ld-linux-x86-64.so.2
00007f4a7ba8f3000   188K  r-x--   ld-linux-x86-64.so.2
00007f4a7ba920000    44K  r----   ld-linux-x86-64.so.2
00007f4a7ba92d000     8K  r----   ld-linux-x86-64.so.2
00007f4a7ba92f000     4K  rw---   ld-linux-x86-64.so.2
00007ffd2f630000    132K  rw---   [ stack ]
total             2768K
```

Observation

The /proc//maps file shows the virtual memory regions assigned to a running process. Common mappings include:

- **Executable/text region** — contains the program's machine instructions.
- **Read-only region** — contains read-only data and other protected mappings.
- **Data region** — contains initialized and writable program data.
- **Heap** — used for dynamically allocated memory.

• **Shared libraries** — libraries such as the C library are mapped into the process's virtual address space.

• **Stack** — contains local variables and function-call information.

• **VDSO/VVAR** — special kernel-provided mappings used for certain system operations and timing-related functionality.

The permissions shown in /proc//maps describe what operations are allowed:

r = read

w = write

x = execute

p = private mapping

Result

The memory organization of a running Linux process was successfully studied using /proc//maps, /proc//status, /proc//smaps, and pmap. The experiment demonstrated how virtual memory mappings divide a process's address space into code, data, heap, stack, libraries, and other regions.s = shared mapping